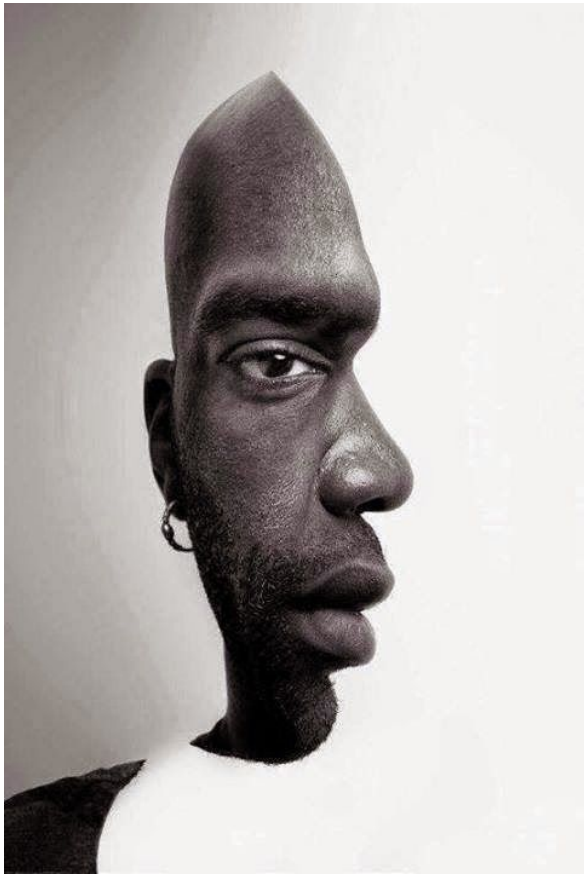
Convolutional Networks

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

What do you see?

Image_dim = H,W,C

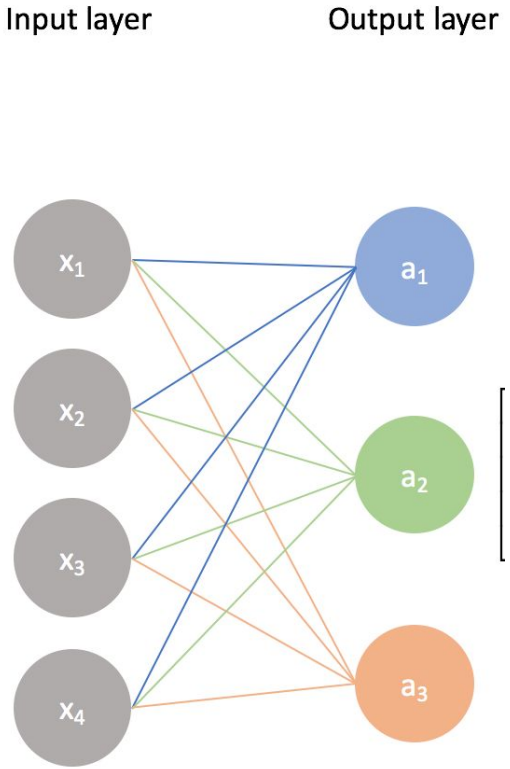
MNIST = ?



0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	8	38	137	146	232	254	255	255	197	109	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	87	197	253	253	253	253	253	253	253	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	120	237	253	253	253	248	209	139	139	230	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	112	229	210	128	96	0	0	0	0	117	253	188	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	28	0	0	0	0	0	0	45	241	245	82	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	60	253	125	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	32	134	148	98	127	4	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	7	201	253	200	59	12	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	122	246	253	223	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	233	253	253	236	55	11	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	233	253	253	253	253	210	48	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	68	193	185	243	253	253	173	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	59	253	253	226	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	22	253	253	226	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	10	93	253	253	123	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	14	180	253	253	228	42	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	95	181	253	253	253	66	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	6	33	33	100	178	253	253	253	241	88	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	32	236	253	253	253	253	253	228	122	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	143	253	253	253	154	76	24	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

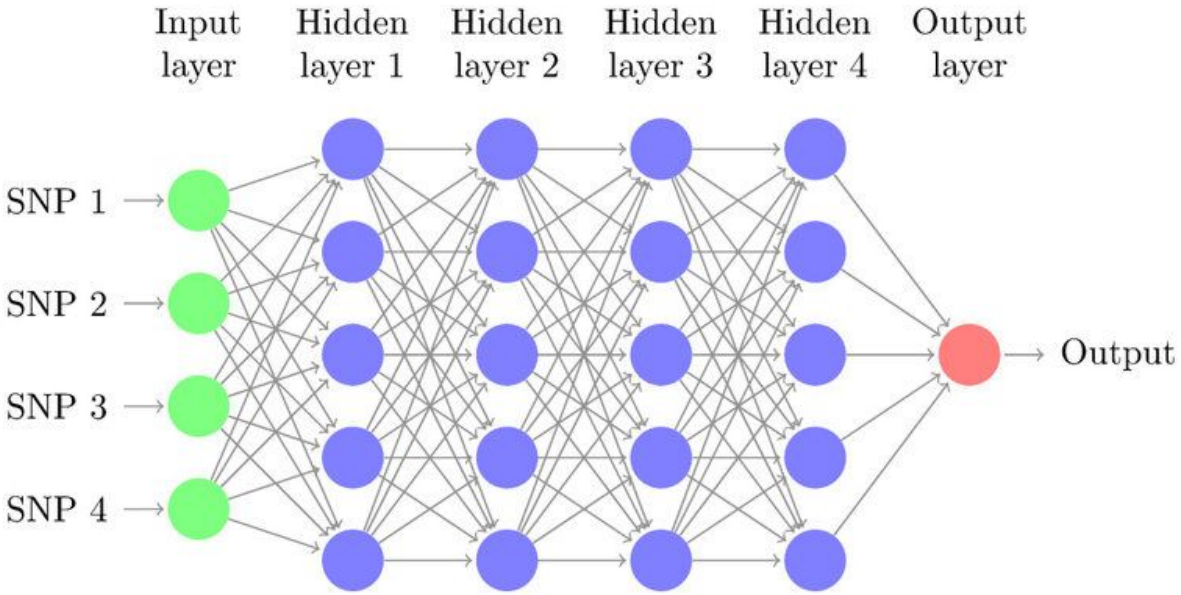
MLP: Recap

- **Recursively apply function mapping**



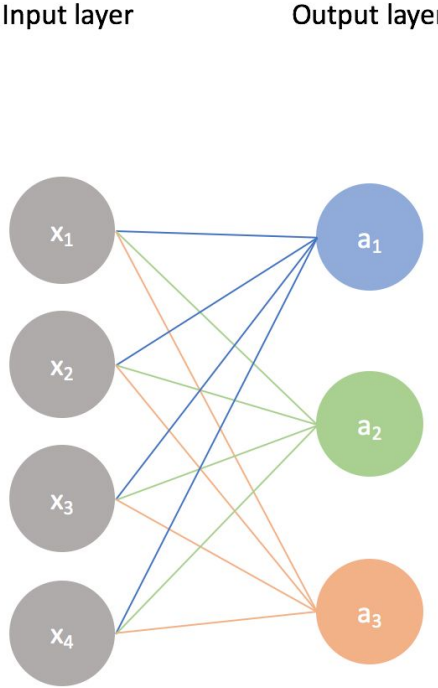
MLP: Recap

Unrolled



MLP: Recap

OUT = **Func**(inp,learnable_paramters):



A simple neural network

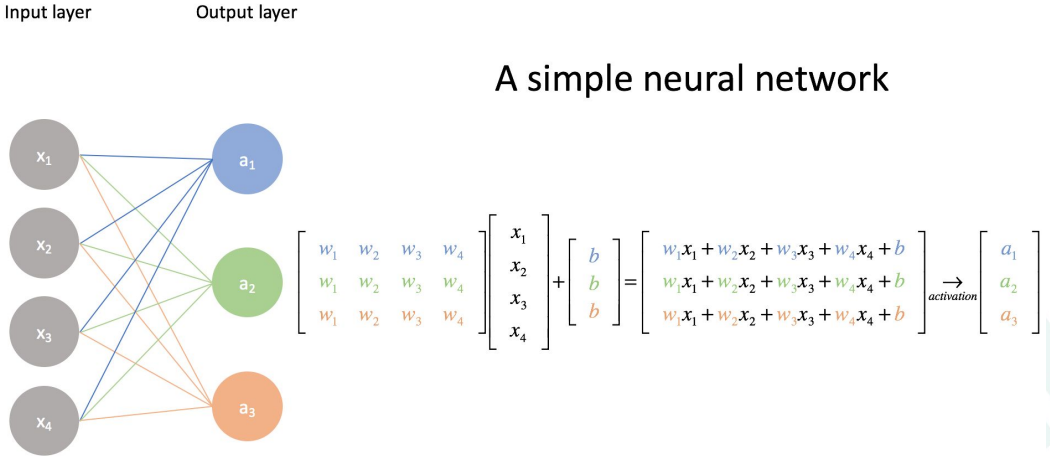
$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow{activation} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

MLP: Recap

OUT = **Func**(inp,learnable_paramters):

Number of learnable parameters in a layer given the input and output are of shape A and B respectively?

- **?**

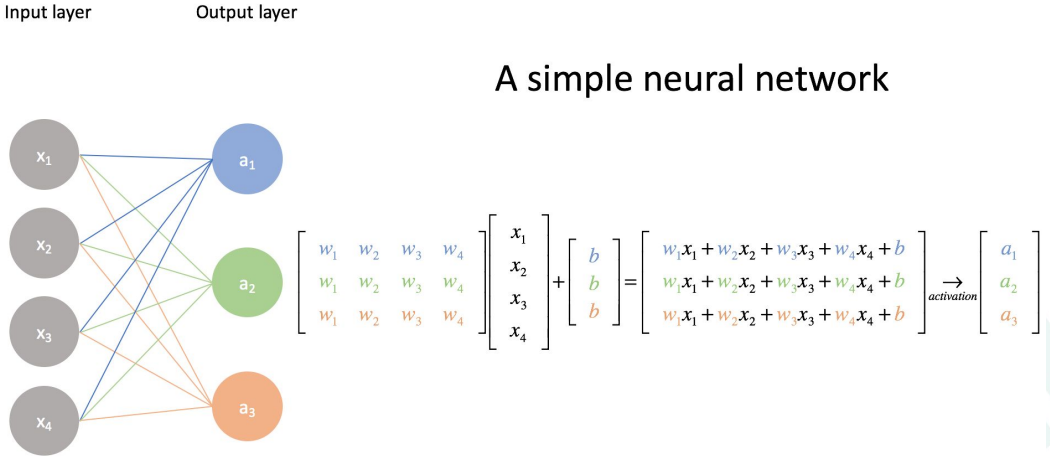


MLP: Recap

OUT = **Func**(inp,learnable_paramters):

Number of learnable parameters in a layer given the input and output are of shape A and B respectively?

- A*B + constant (bias)
- If A = 784 and B = 10. What is A*B?

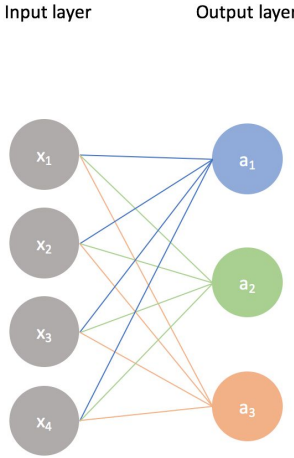


MLP: Recap

OUT = **Func**(inp,learnable_paramters):

Number of learnable parameters in a layer given the input and output are of shape A and B respectively?

- A*B + constant (bias)
- If A = 784 and B = 10. What is A*B?
- If A = 28*28*1 and B = 10. What is A*B?
- If A = 224*224*3 and B = ?
- If A = 224*224*3 and B = ?. What is A*B



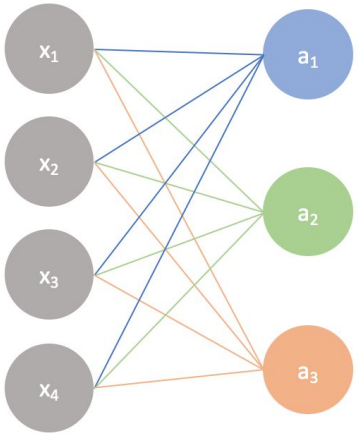
A simple neural network

$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow{activation} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

MLP: Properties

- Number of parameters grow proportionally to the input size. Bcoz we use models which are deeeeeeeeep.
  - E.g. 224 x 224 x 3 image:

- **Not** translation invariant: Reacts differently to an input's shifted version
  - Inp1 = [1,1,2,2]
  - Inp2 = [2,2,1,1]



$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow{activation} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

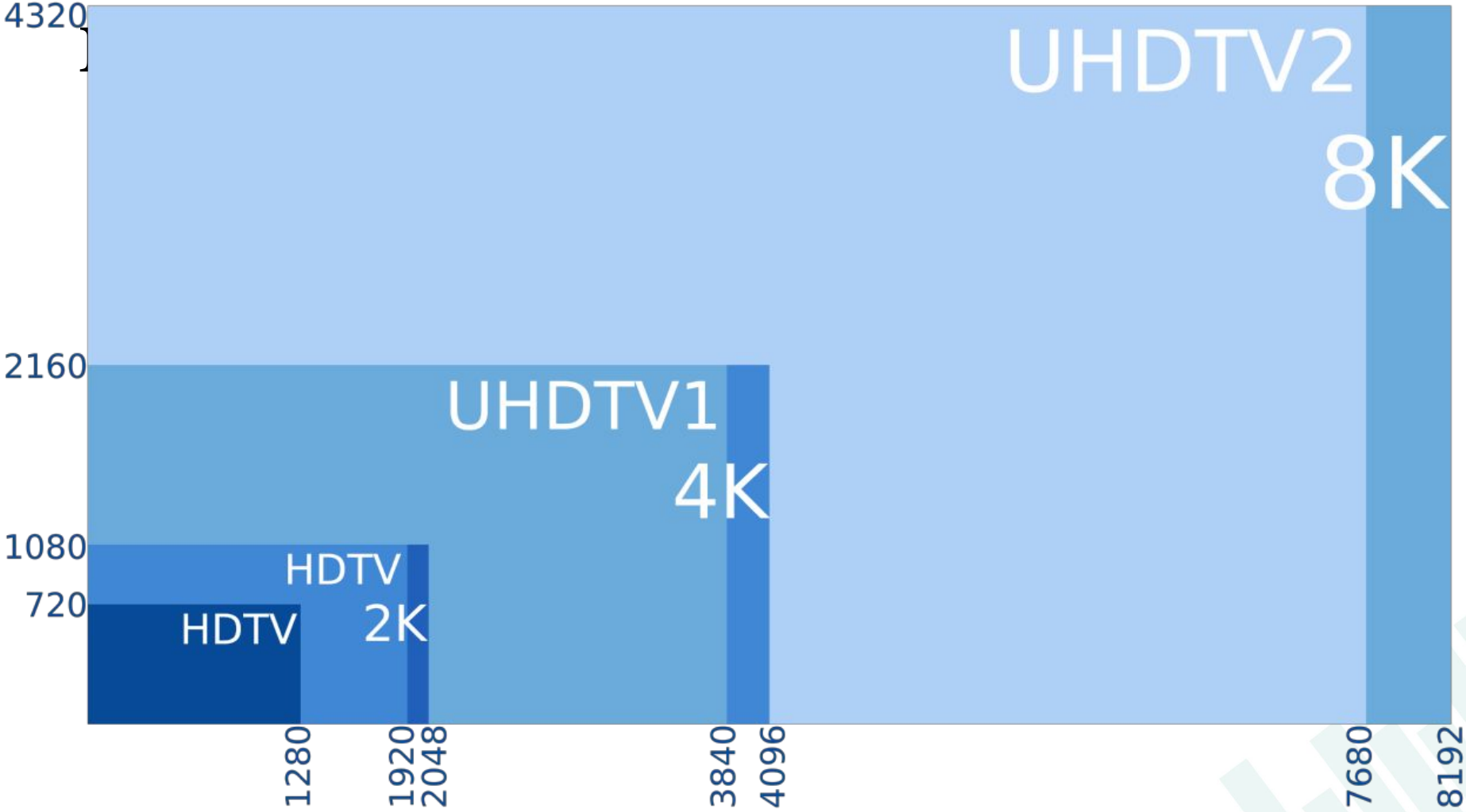
Desirable properties given the input is **IMAGE**

The function has/is?

1. Learnable parameters should not grow with the input size.

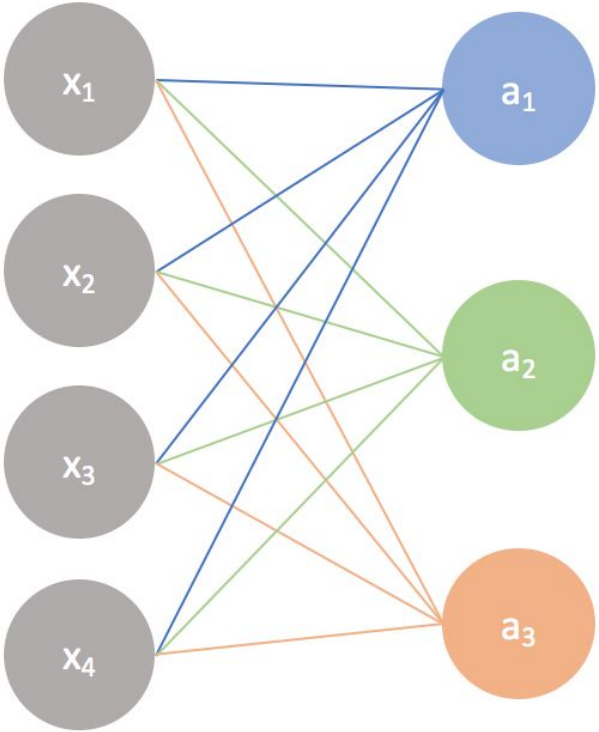
    a. Size of a HD image = 1280 x 720 x 3 and B = 10. ?

2. Translation **IN**variant:


4320
2160
1080
720
UHDTV2
8K
UHDTV1
4K
HDTV
2K
HDTV
1280
1920
2048
3840
4096
7680
8192

Possible solution

Number of learable parameters?

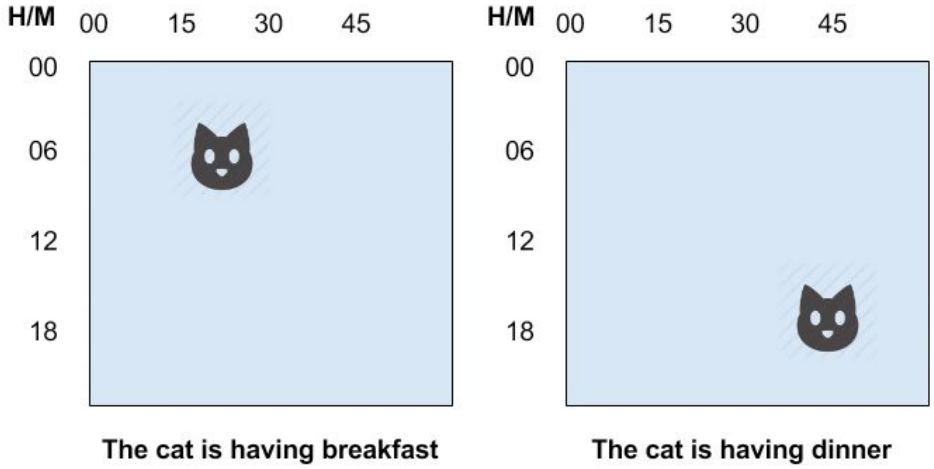


$$\begin{bmatrix} w_1 & w_2 & w_1 & w_2 \\ w_1 & w_2 & w_1 & w_2 \\ w_1 & w_2 & w_1 & w_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

MLP: Solution

1. Learnable parameters should not grow with the input size?

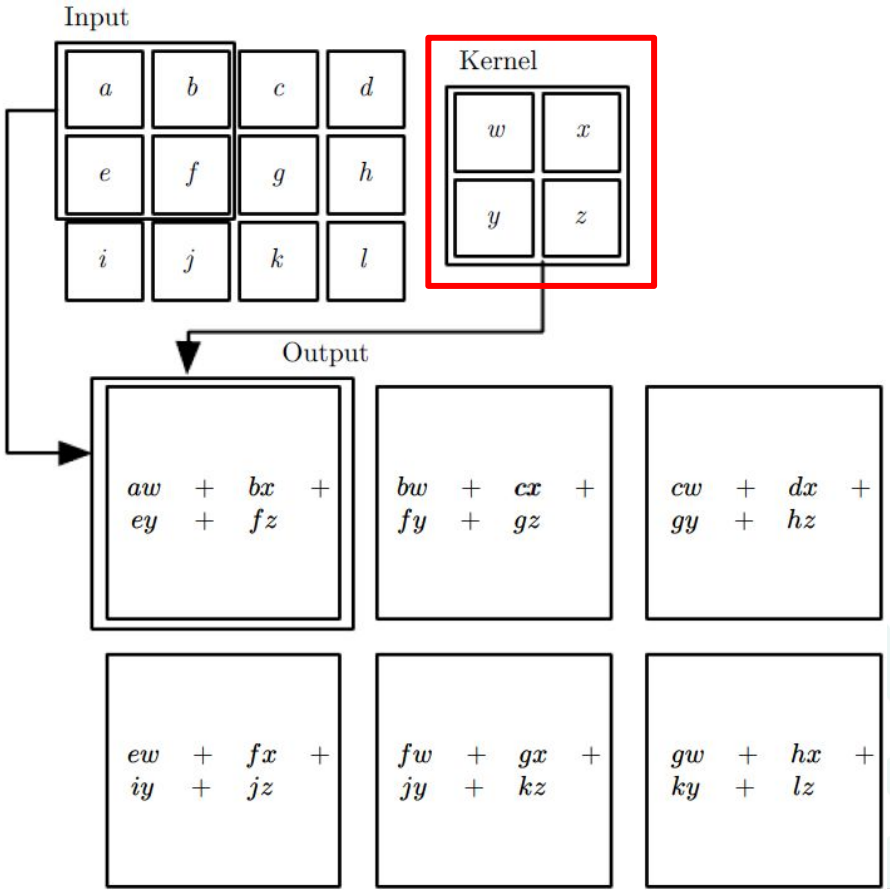
2. Translation **IN**variant: Local information can move in 2 dimension.



Convolution Operation: element wise dot product

Stride = 1

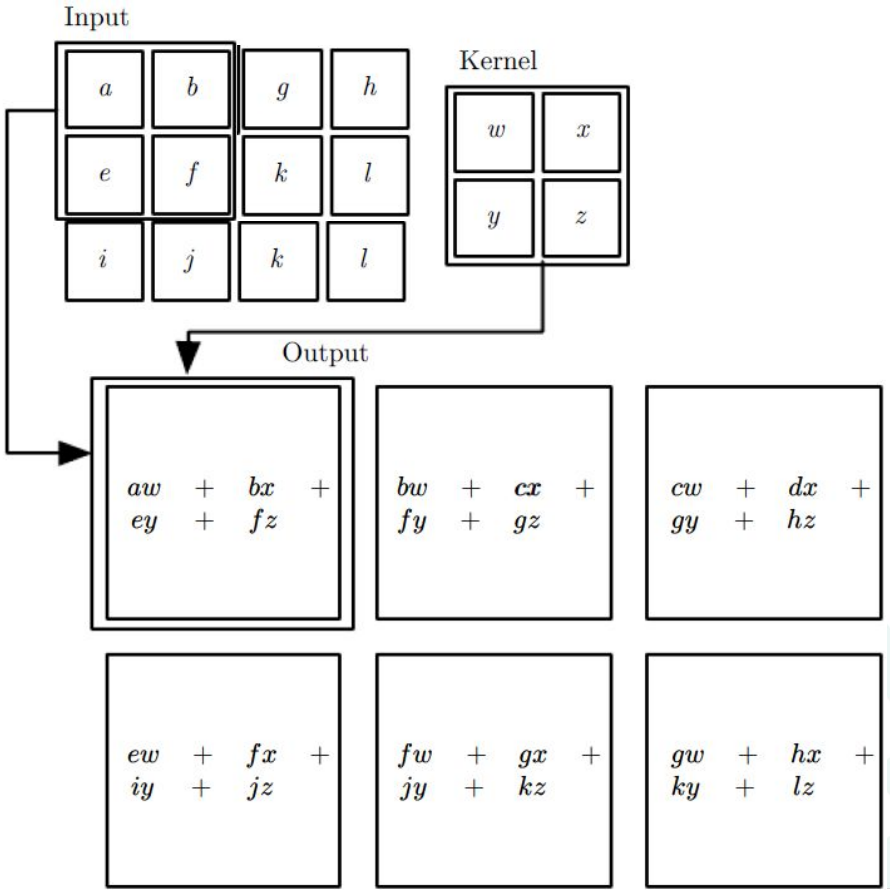
Input independent parameters: Tick



Convolution Operation: element wise dot product

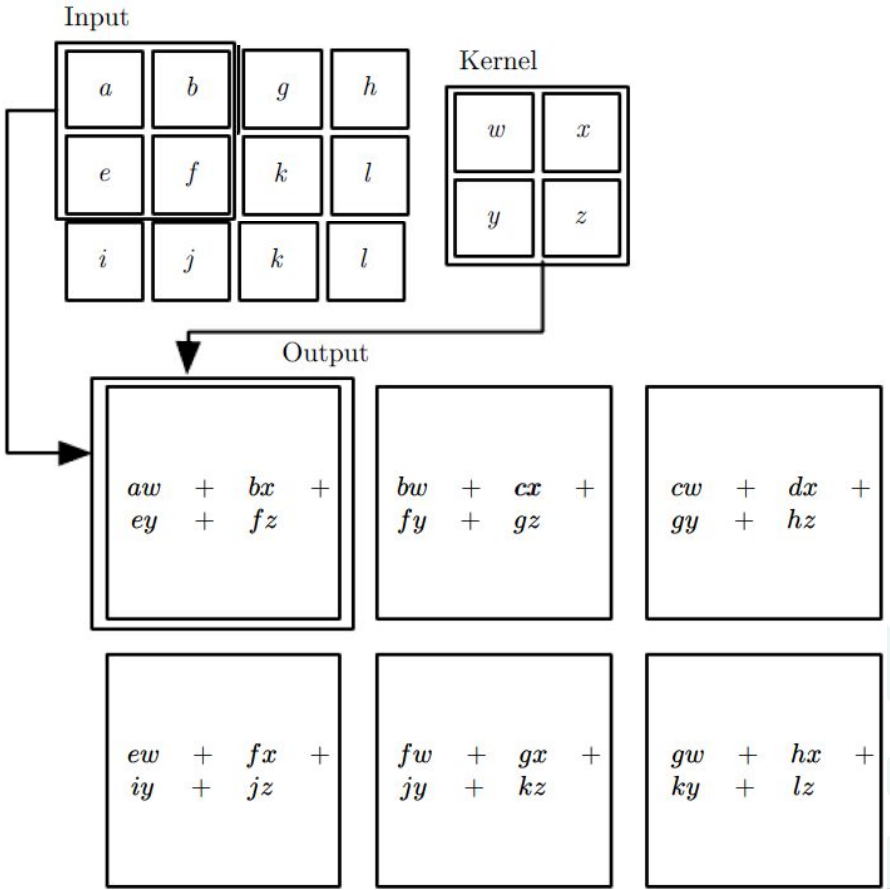
Stride = 1

Translation invariant: Tick



Convolution Operation: element wise dot product

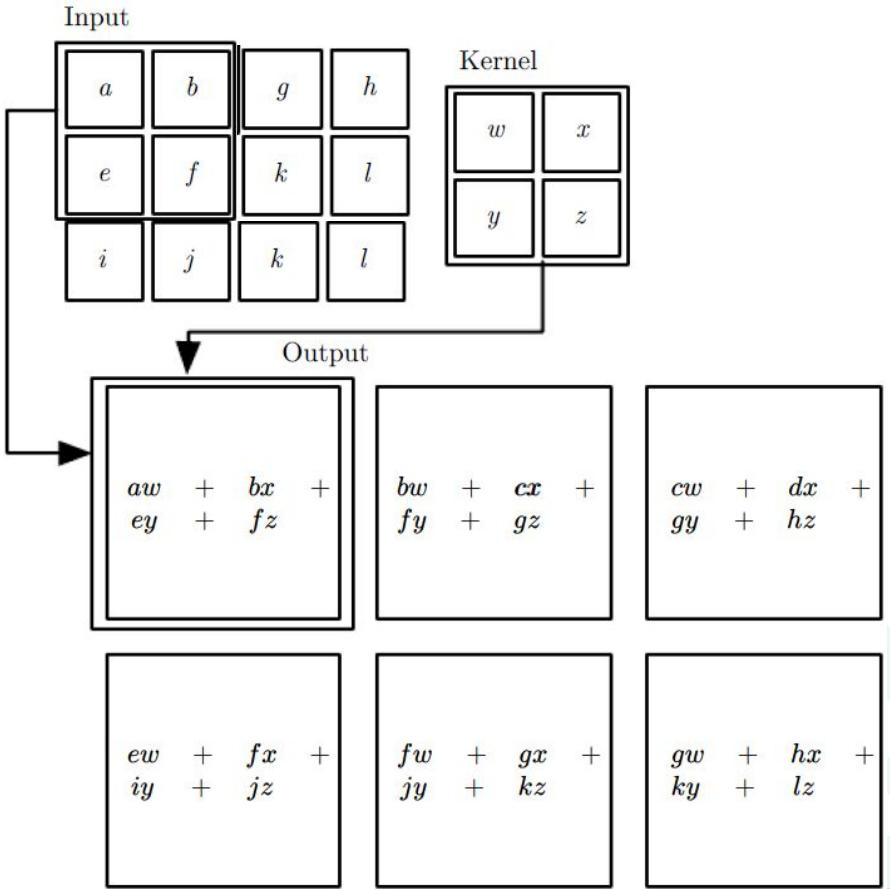
What is the output image dimension?



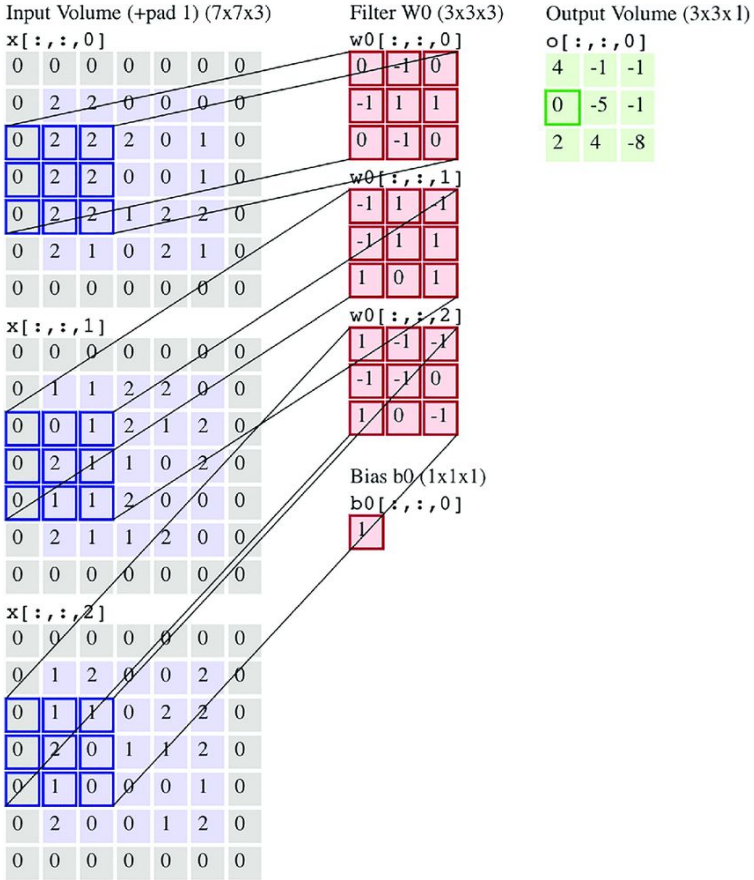
Convolution Operation: element wise dot product

Property:

- output size is reduced.
- Depends on the stride.

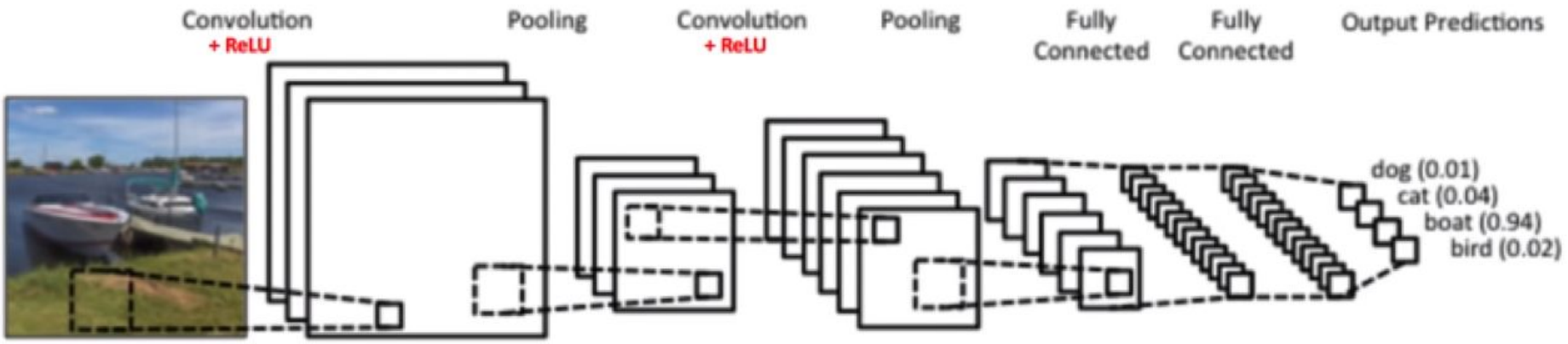


What is the stride?

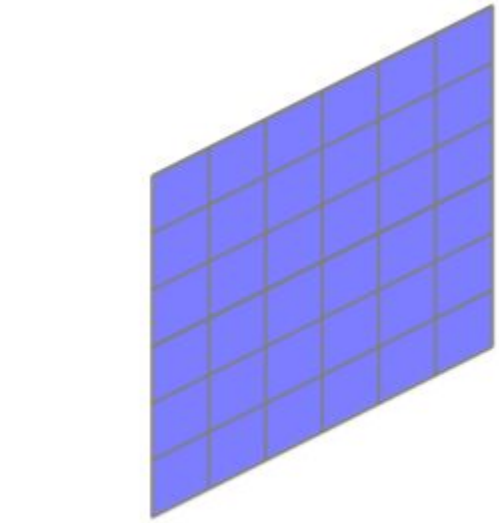


- Input has 3 channels (h,w,c) that is why 3 kernels.
- But the output has only 2 channels (h,w,1).

CNN: Architecture



Visual example: Convolution Function

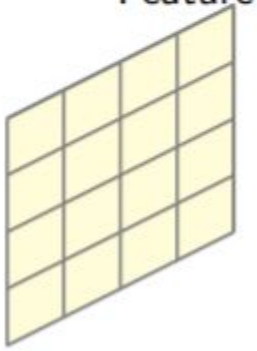


Grayscale Image

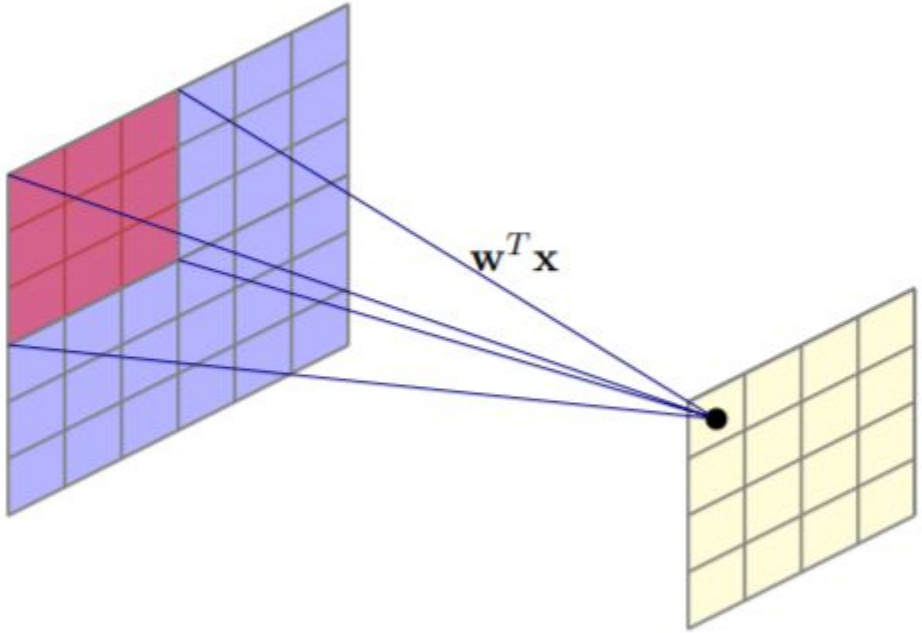
Kernel

w_7	w_8	w_9
w_4	w_5	w_6
w_1	w_2	w_3

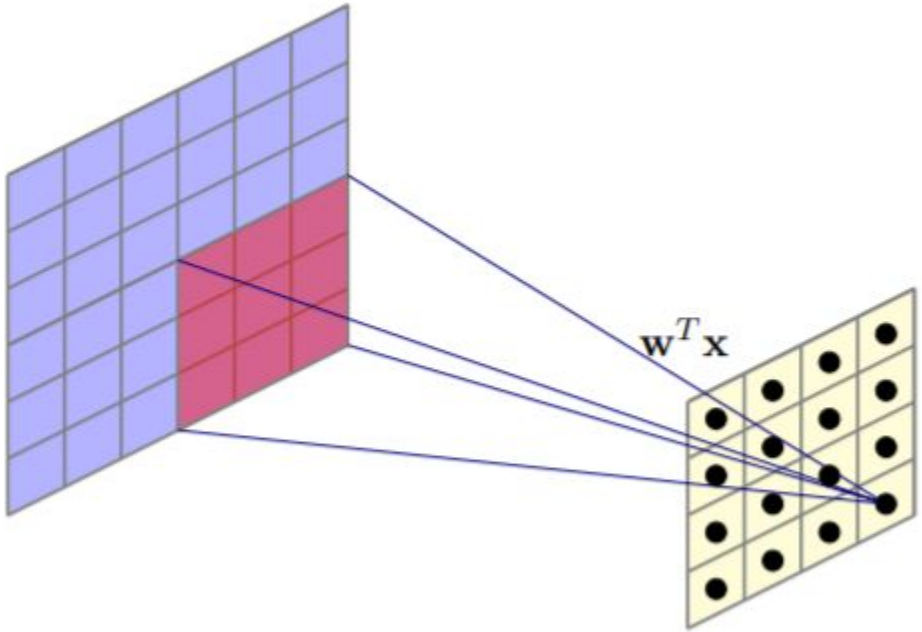
Feature Map



Visual example example: Convolution Operation

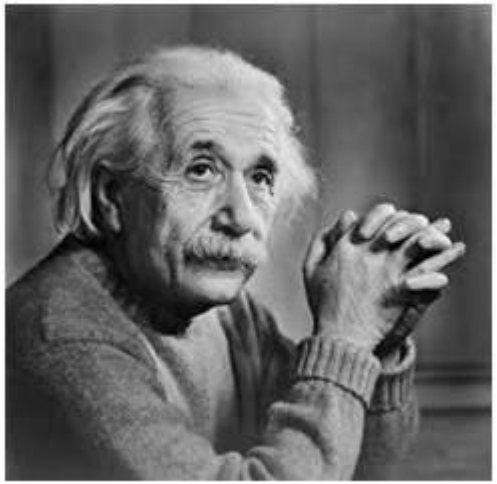


An example: Convolution Operation



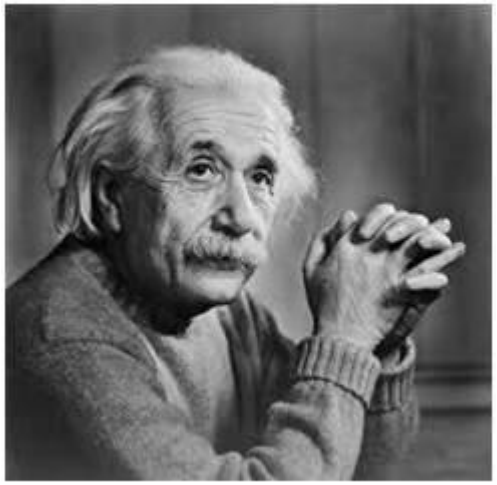
Convolution: Vertical

-1	0	1
-1	0	1
-1	0	1



Convolution: Horizontal

-1	-1	-1
0	0	0
1	1	1



Combined

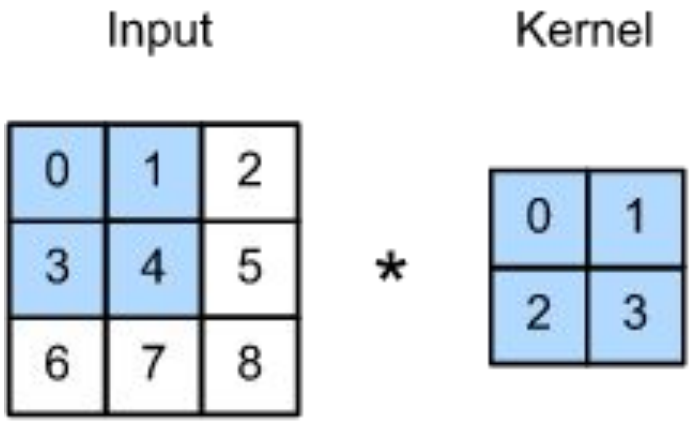
Convolution Operation: Output Size

- For convolutional layer:
  - Suppose input is of size $W_1 \times H_1 \times D_1$
  - Filter size is K and stride S
  - We obtain another volume of dimensions $W_2 \times H_2 \times D_2$
- As before:
  - $W_2 = (W_1 - K)/S + 1$ and $H_2 = (H_1 - K)/S + 1$
- Depths will be equal ? That depends…on the number of kernelsssss…..

Convolution Operation: Output Size

- Output size: (N – K)/S + 1
  - N: input dimension
  - K: Kernel size
  - S: Stride
- In previous example:
  - N = 6, K = 3, S = 1,
  - Output size =
    - 4

Quiz 1



What would be the output of the cell (2x2) convolution operation?

1. 44
2. 45
3. 41
4. 42

Convolution: Motivation

- Convolution leverages four ideas that can help ML systems:
  - Sparse interactions
  - Parameter sharing
  - Translation invariant.
  - Ability to work with inputs of variable size

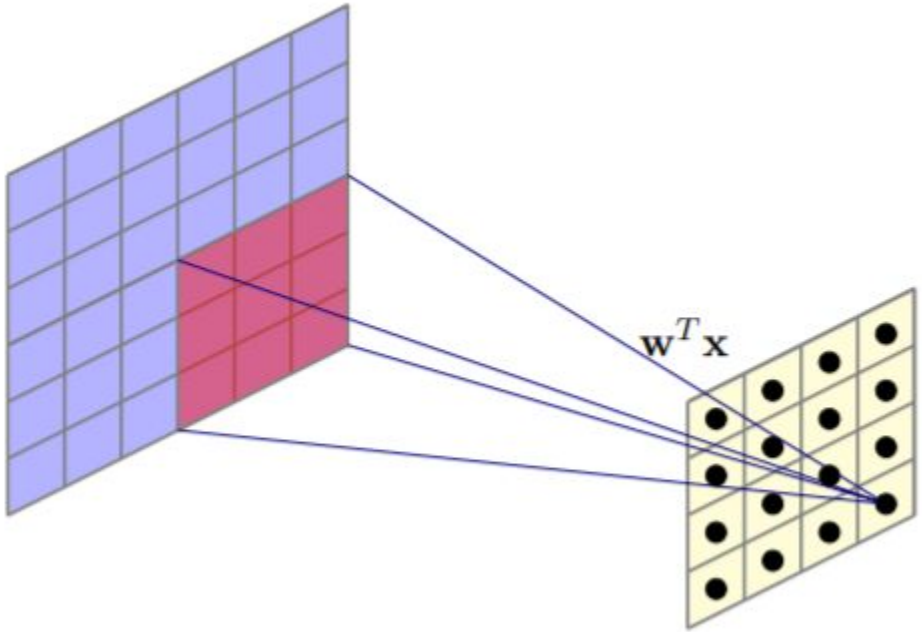
Convolution: Motivation

- Sparse interactions

Sparse interactions in Convolutional Neural Networks (CNNs) refer to the idea that not all neurons or units in a layer are connected to every neuron in the previous layer. Instead of dense connections, where each neuron is connected to every neuron in the previous layer, sparse interactions involve selectively connecting neurons with a specific spatial arrangement. This is typically achieved through the use of convolutional layers in CNNs.

In a convolutional layer, each neuron is connected to a local region of neurons in the previous layer, and these connections are shared across the entire layer. This sparsity of connections helps reduce the number of parameters in the network and enforces a form of translational equivariance, meaning that features learned by the network can be applied at multiple spatial locations within the input data. Sparse interactions are a key component of the efficiency and effectiveness of CNNs in tasks such as image processing and computer vision.

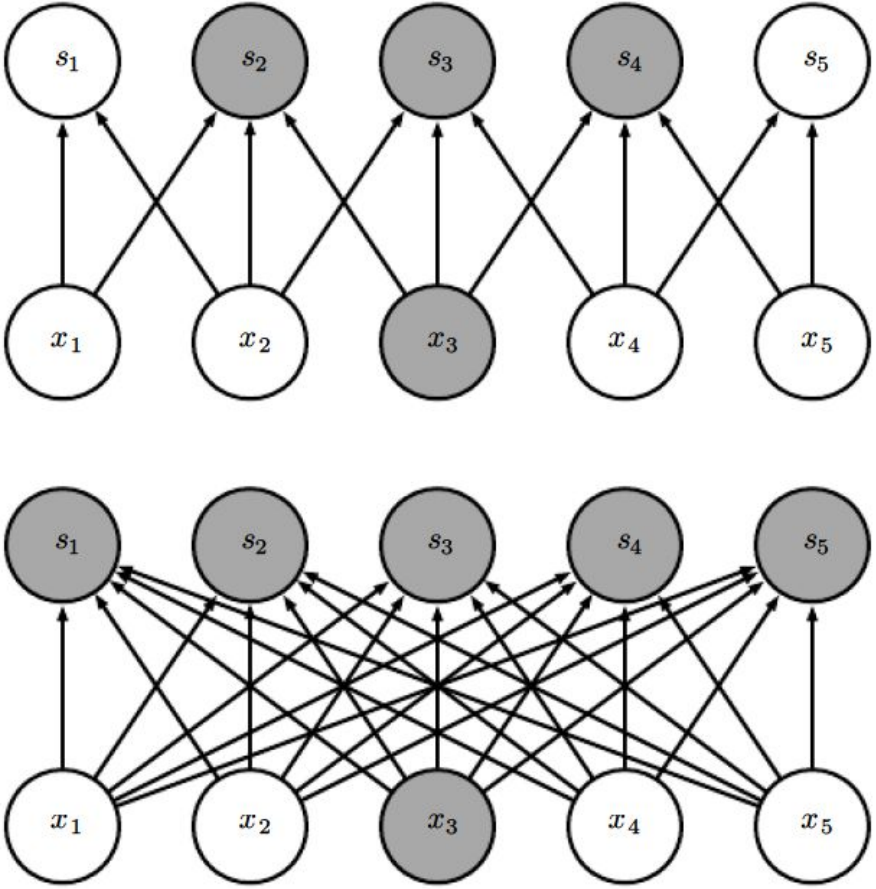
Sparse connections and parameter sharing



Convolution Operation: Sparse Interactions

- This is accomplished by making the kernel smaller than the input.
  - need to store fewer parameters, computing output needs fewer operations ($O(m \times n)$ versus $O(k \times n)$)

Convolution Operation: Sparse Interactions



Quiz 2

Why is it useful to perform parameter sharing in convolution networks?

1. Reduce the computational needs
2. Control the number of parameters
3. Both a and b
4. None of the above

Convolution Operation:Equivariant representations?

- Images: If we move an object in the image, its representation will move the same amount in the output
- This property is useful when we know some local function is useful everywhere (e.g. edge detectors)
- Convolution is not equivariant to other operations such as change in scale or rotation
- It is Translational invariant.

Input = 2d matrix

1. Max
2. Min
3. Average
4. ?

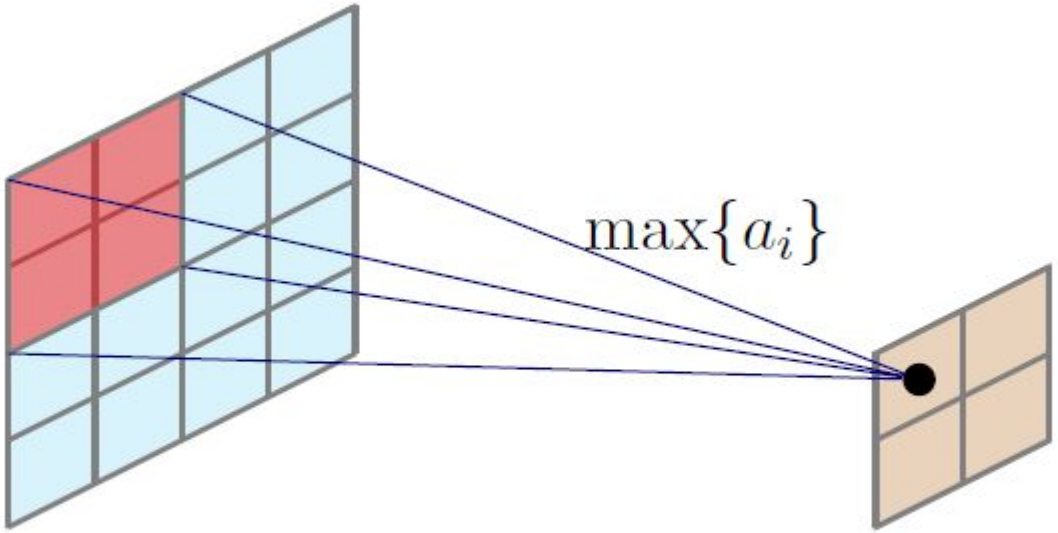
- A function which maps N values to 1 is called pooling in the CNN context.
- Mathematical property is to decrease the dimension.

```
[[1., 1.],
[1., 1.]]
```

Pooling

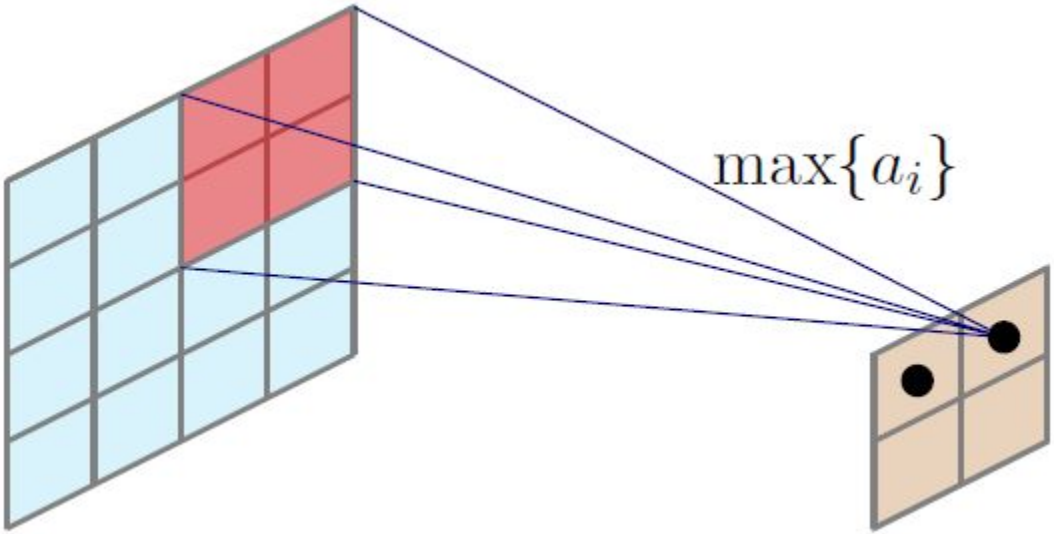
- A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs.
  - Max pooling operation reports the maximum output within a rectangular neighborhood.
- Other popular pooling functions include
  - the average of a rectangular neighborhood,
  - the L2 norm of a rectangular neighborhood, or
  - a weighted average based on the distance from the central pixel
- In all cases, pooling helps to make the representation become approximately invariant to small translations of the input.

Pooling: Max Pooling

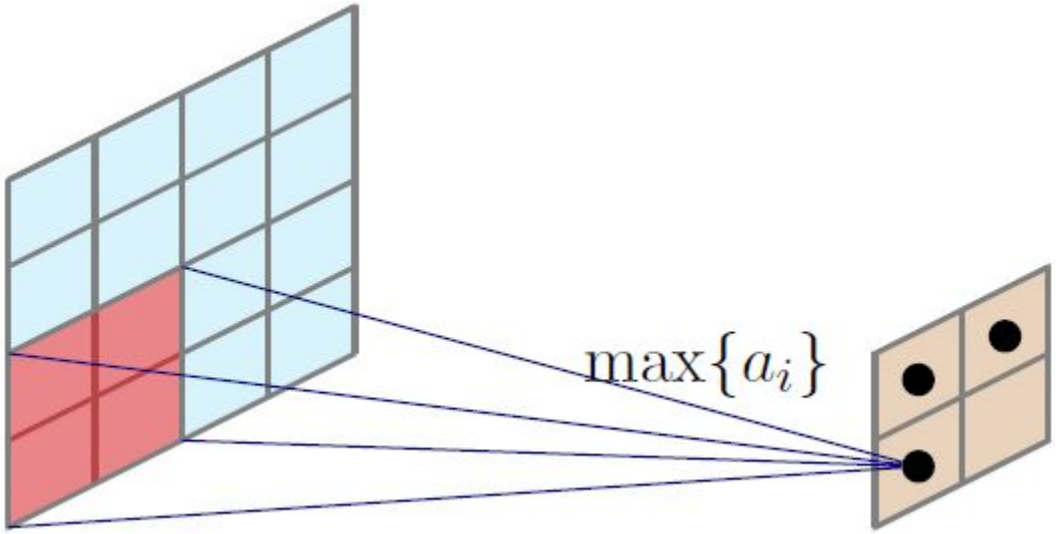


Think if polling similar to convolution.

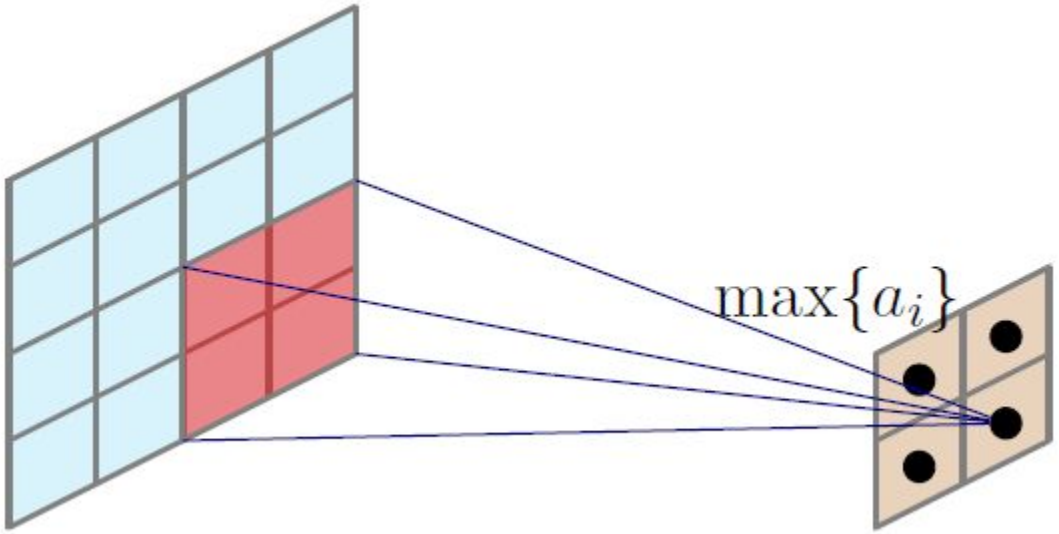
Pooling: Max Pooling



Pooling: Max Pooling

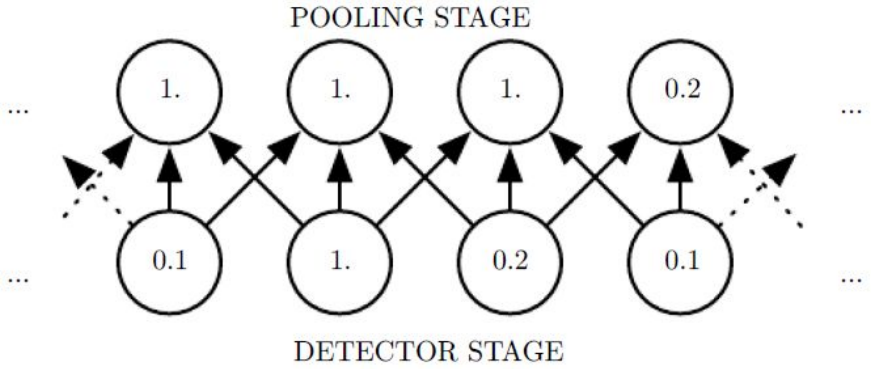


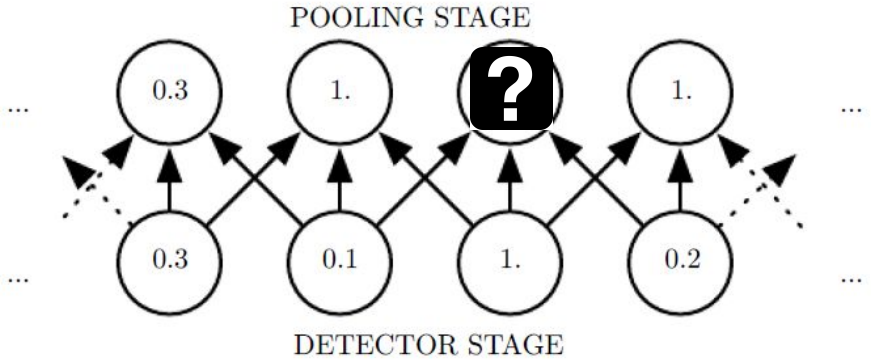
Pooling: Max Pooling



What is the stride?

Pooling

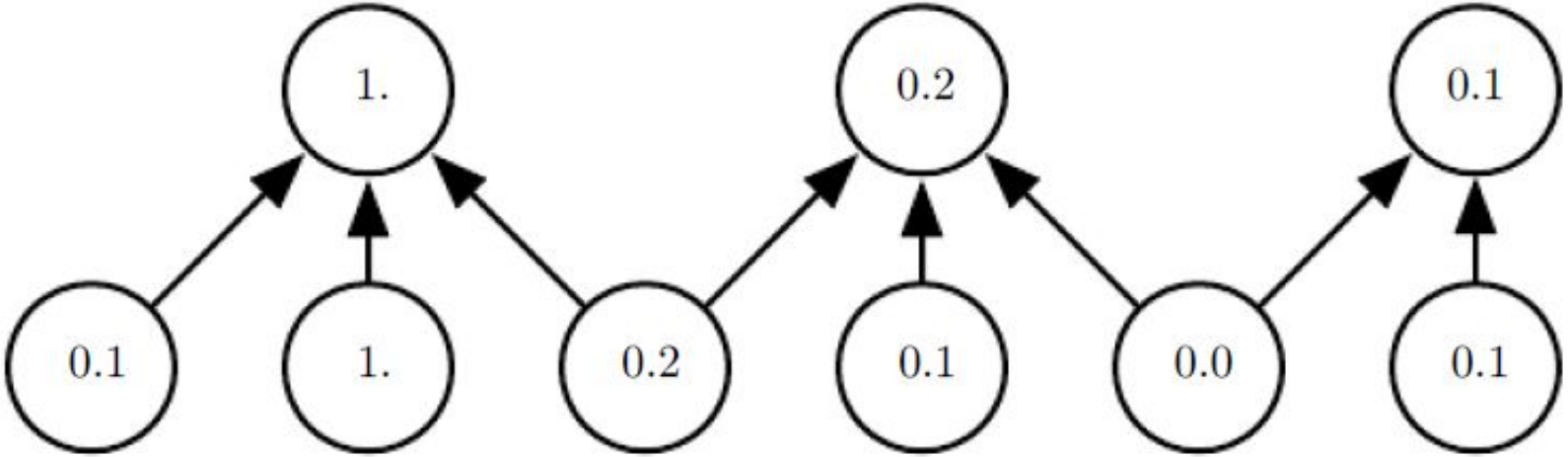

POOLING STAGE
1.
1.
1.
0.2
0.1
1.
0.2
0.1
DETECTOR STAGE


POOLING STAGE
0.3
1.
?
1.
0.3
0.1
1.
0.2
DETECTOR STAGE

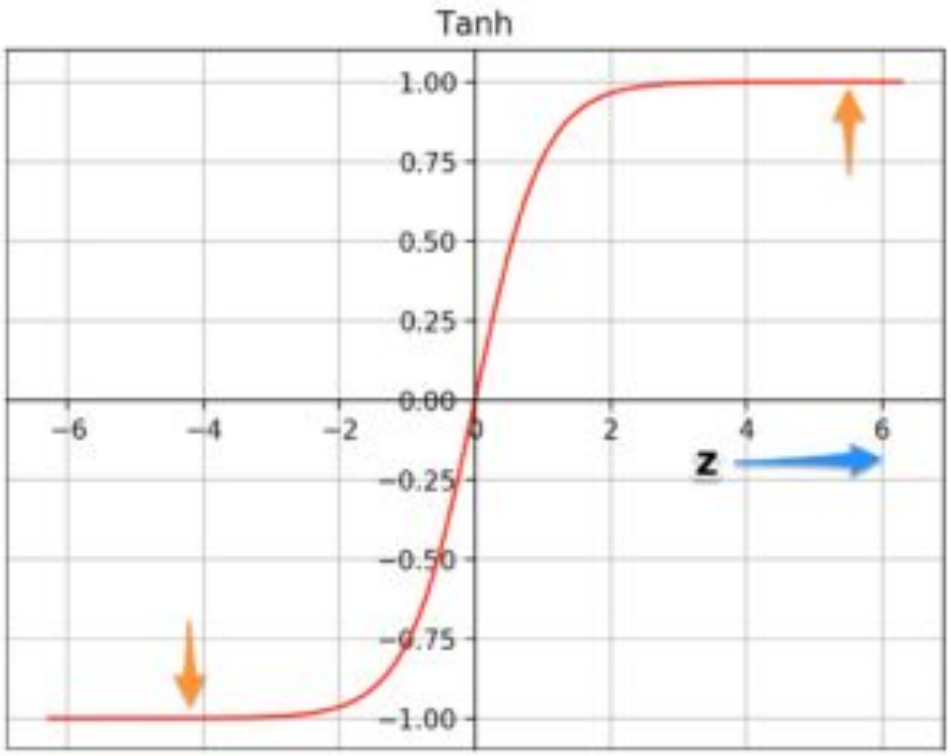
Pooling

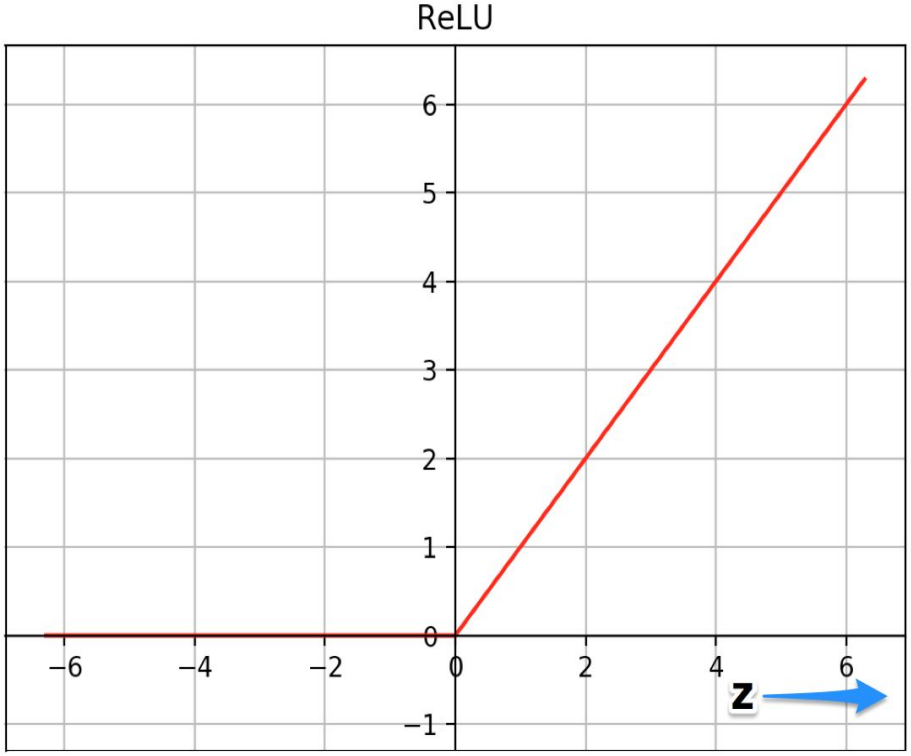
- Invariance to local translation can be a very useful property if we care more about *whether some feature is present* than *exactly where it is*
  - For example, when determining whether an image contains a face, we need not know the location of the eyes with pixel-perfect accuracy.
- In other contexts, it is more important to preserve the location of a feature.
  - For example, if we want to find a corner defined by two edges meeting at a specific orientation
- Since pooling is used for downsampling, it can be used to handle *inputs of varying sizes*

Pooling: Downsampling

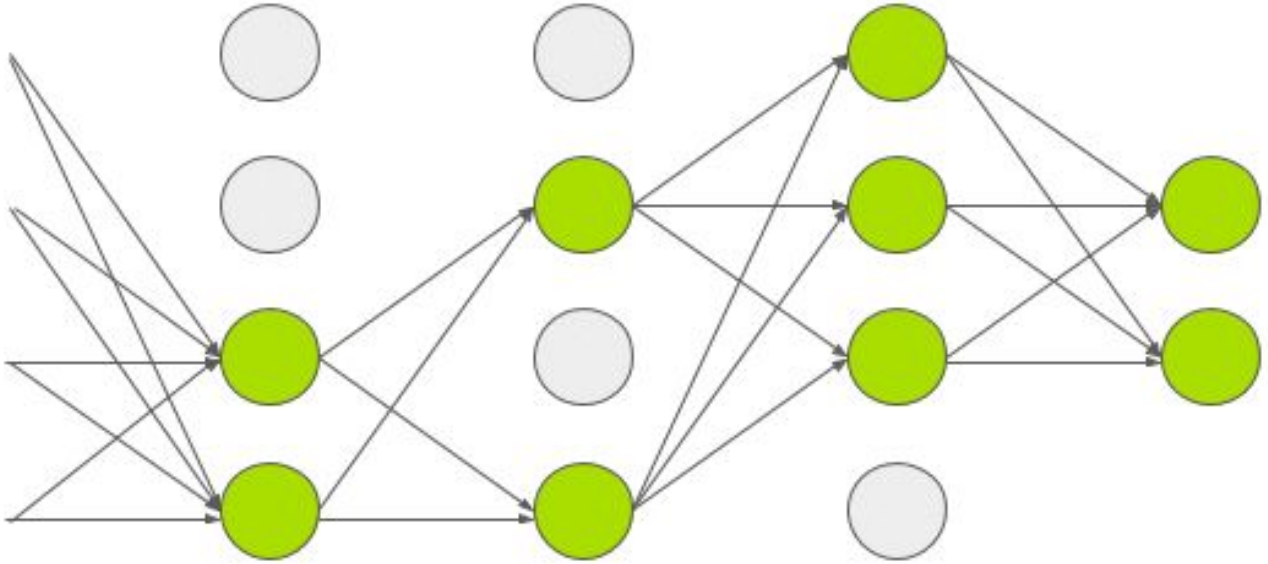


ReLu: Rectified Linear Unit

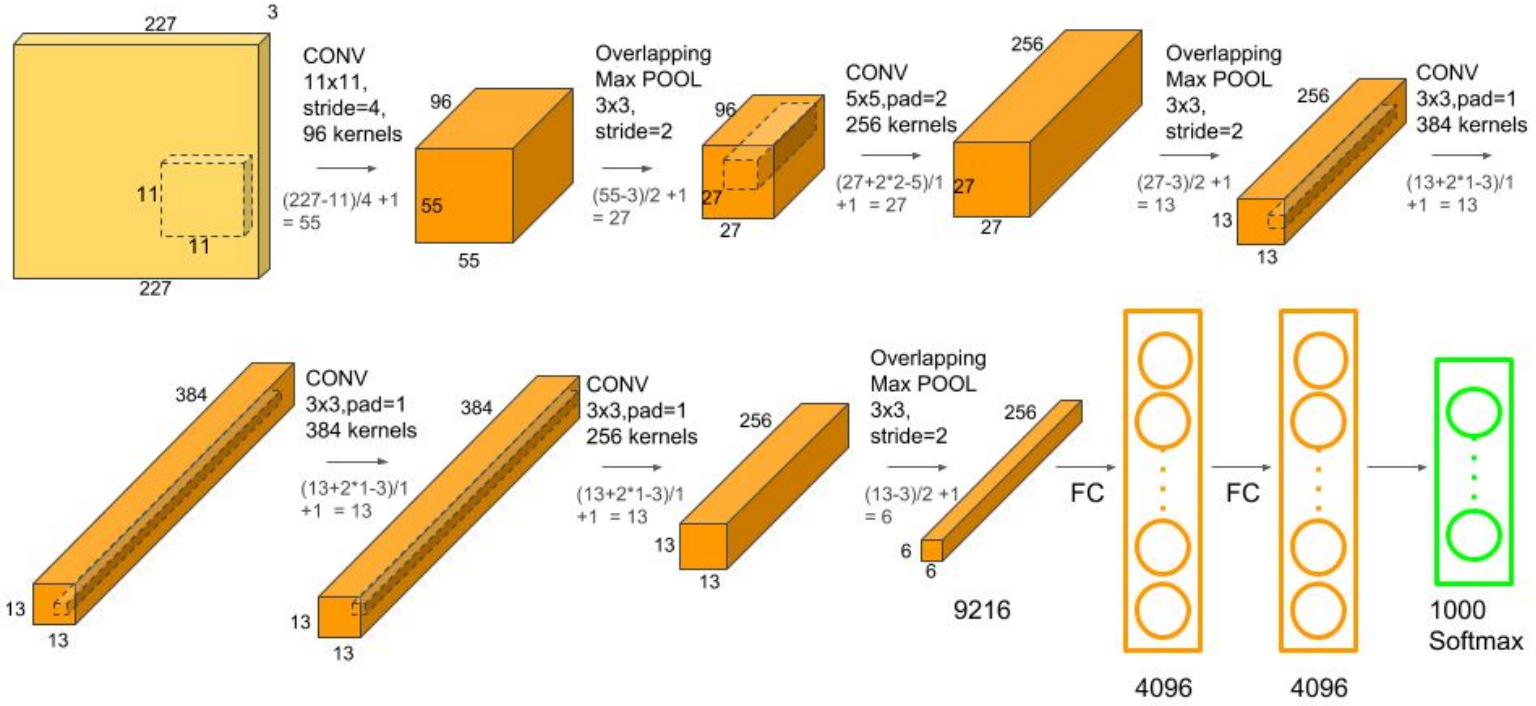




Dropout: Overfitting.



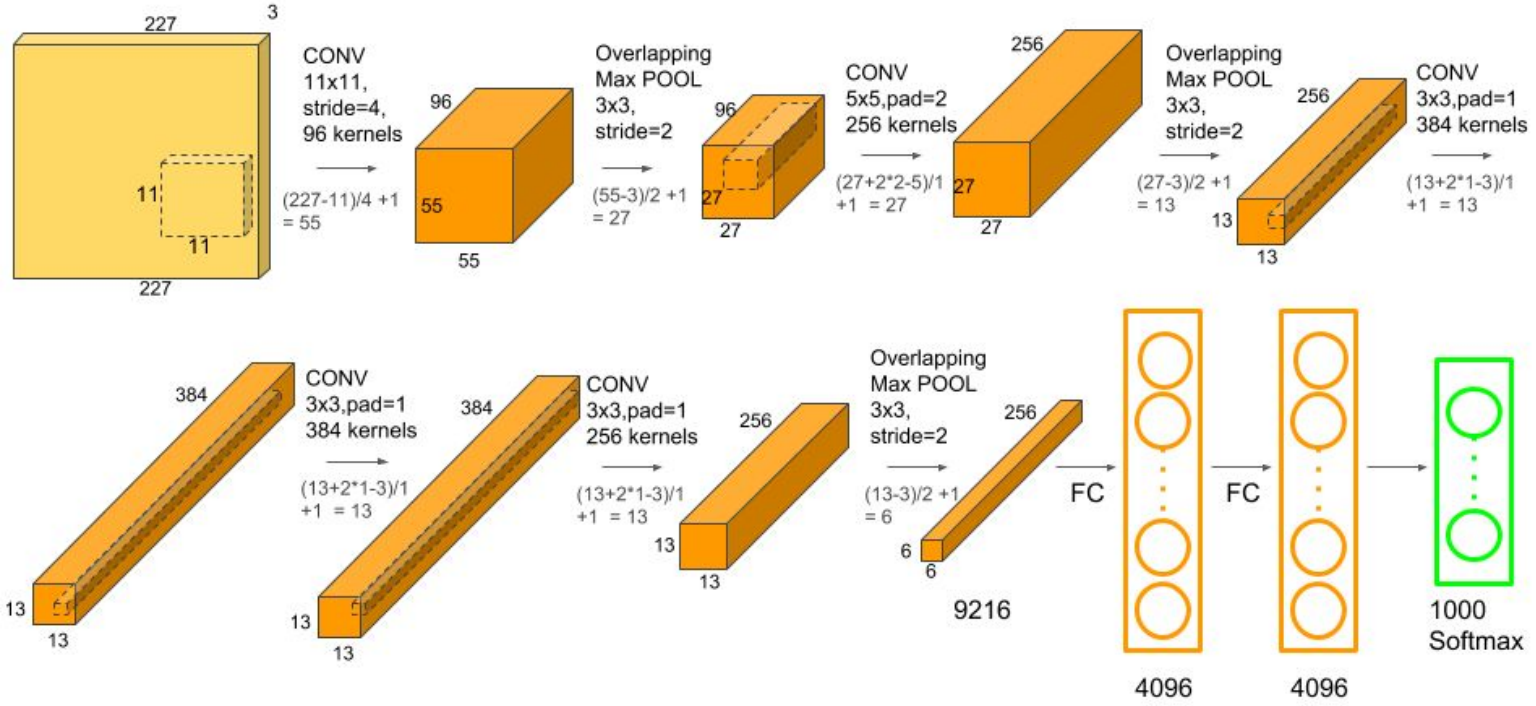
Walk Through: AlexNet



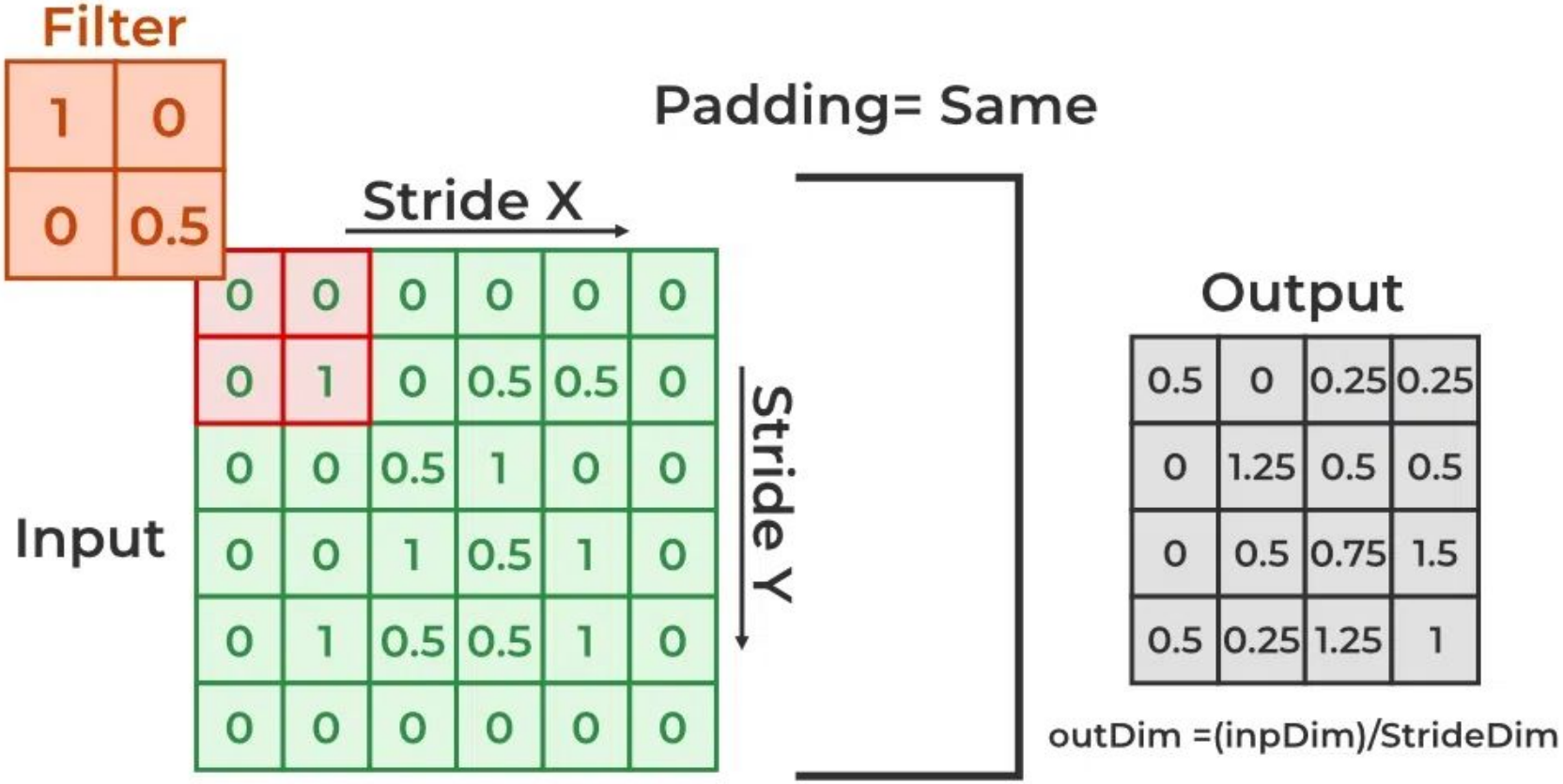
Walk Through: AlexNet

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

Walk Through: AlexNet

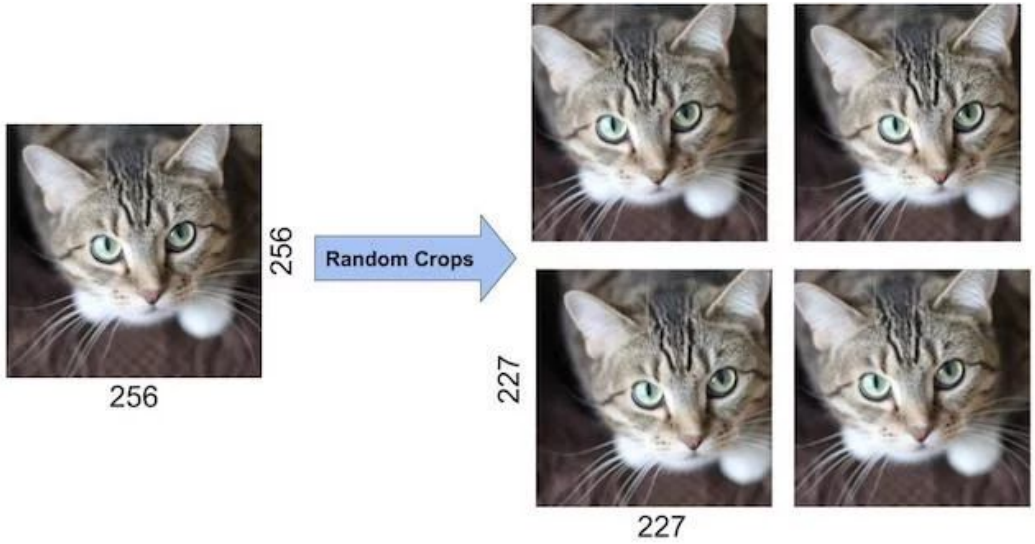


A short course on Padding in the context of images



AlexNet: Data augmentation

- The input to AlexNet is an RGB image of size 256×256×3
- Random crops of size 227×227×3 were generated from inside the 256×256×3 images to feed the first layer of AlexNet.
- Input: 227×227 x 3



Quiz 3

You have an input volume that is 15x15x8, and pad it using "pad=2." What is the dimension of the resulting volume (after padding)?

1. 19x19x12
2. 17x17x8
3. 19x19x8
4. 17x17x10

AlexNet

- 5 Convolutional Layers and 3 Fully Connected Layers
- It has 60 million parameters and 650,000 neurons and took five to six days to train on two GTX 580 3GB GPUs!
- Input: 227×227 x 3
- The first Conv Layer of AlexNet contains 96 kernels of size 11x11x3.
  - Width and height of output: (227-11)/4 + 1 = 55
- Number of parameters in first layer?
  - (11x11x3 +1)*96 = 34,9444
- Number of Computations: 34,9444 x 55 x 55 = 10,57,05,600
- https://airtable.com/shrArXKRCau4KhAwZ/tbloN5WYjFYpKGUEt?blocks=hide

Desirable properties given the input is **IMAGE**

The function has/is?

1. Learnable parameters should not grow with the input size.
    a. Size of a HD image = 1280 x 720 x 3 and B = 10. ?

2. Translation **IN**variant:

3. Better if we could get rotational invariant or scale invariant?

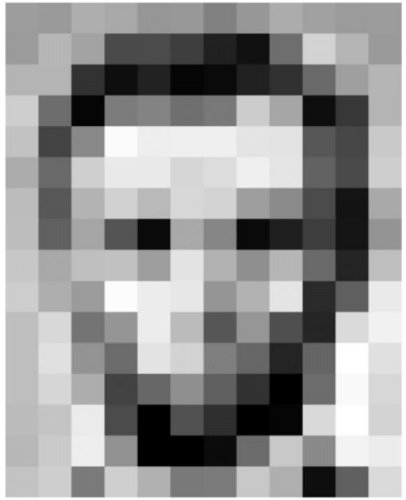
Training on the left and testing on the right won’t work.

Why?

Hint: kernel size.

What computers 'see': Images as Numbers

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

What the computer "sees"

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

Levin *Image Processing & Computer Vision*

An image is just a matrix of numbers [0,255]. i.e., 1080x1080x3 for an RGB image.
Question: is this Lincoln? Washington? Jefferson? Obama?
How can the computer answer this question?

Can I just do classification on the 1,166400-long image vector directly?

No. Instead: exploit image spatial structure. Learn patches. Build them up

References

1. Chapter 9, Deep Learning: Ian Goodfellow, Yoshua Bengio, Aaron Courville (http://www.deeplearningbook.org/)
2. https://neurdiness.wordpress.com/2018/05/17/deep-convolutional-neural-networks-as-models-of-the-visual-system-qa/
3. https://www.rctn.org/bruno/public/papers/Fukushima1980.pdf
4. https://www.youtube.com/watch?v=2-Ol7ZB0MmU
5. https://www.youtube.com/watch?v=ZOXOwYUVCqw